

SIMULATION / ENVIRONMENT / SCENE

Simulation Setup and Scene Construction

Ning Bo 宁博

Student ID 24020036049 | Individual design: simulation blueprint

1 Experiment requirements and individual scope

The experiment develops a fixed-point grasping task without visual localization: home, approach A, descend, grip, lift, transfer to B, release and return. The course requires a complete simulation launch, state publication, saved trajectories/results/errors, joint and reachability checks, and at least four successful transfers in five consecutive trials. Hardware verification additionally requires supervised low-speed operation, safe failure handling and individual safety-operation checks [1].

Because of equipment availability, our group used the DJI RoboMaster EP arm and gripper instead of the specified mechArm 270. The simulation uses ROS 2 Humble, Gazebo Fortress and custom kinematics/control on a Jetson Linux system. The physical implementation uses the RoboMaster Python SDK. This is a platform adaptation, not an unchanged ROS 2 hardware backend.

1.1 My responsibility and team interfaces

My main responsibility is preparing the simulation environment and scene: loading the RoboMaster model, arranging the table and cube, checking resource paths and starting the simulation. I coordinate with Huang Junbiao on model or scene problems that affect his motion-program debugging, and provide the final scene/-configuration material to Qi Yongyue for result analysis. Bao Lintai and Xue Yutong handle the physical experiment. This report focuses on getting a usable simulated setup ready for repeated grasping.

Member	Work area	Main responsibility
Ning Bo	Simulation	Set up the environment and scene; load models and launch the simulation.
Huang Junbiao	Simulation	Work on the motion program and debug grasping, transfer and repeated cycles.
Qi Yongyue	Simulation	Organize logs, analyse results and prepare report material.
	results	
Bao Lintai	Hardware	SDK communication, task sequence, chassis turn/return and exception handling; joint trials.
Xue Yutong	Hardware	Pick/release waypoints, approach/lift heights, gripper adjustment and retention checks; joint trials.

Blueprint note. Scope of evidence. Simulation repeats A→B and verifies home after every cycle; the object is reset only between cycles. Hardware alternates A/B, pauses for human grasp confirmation and returns home after all five cycles. Shared ROS 2 interfaces, parameter-only hardware switching and each member’s safety check are not demonstrated by the supplied records.

2 Scene geometry and physical assumptions

2.1 A common support plane

The world file places the robot and object on a shared table. Its box geometry is $1.6 \times 1.2 \times 0.05$ m, with centre height 0.035 m, giving a top surface at $z = 0.060$ m. The object is a 0.025 m cube, so its resting centre should be $0.060 + 0.0125 = 0.0725$ m. It is spawned at 0.0735 m, 1 mm above that equilibrium height, and settles under the physics engine.

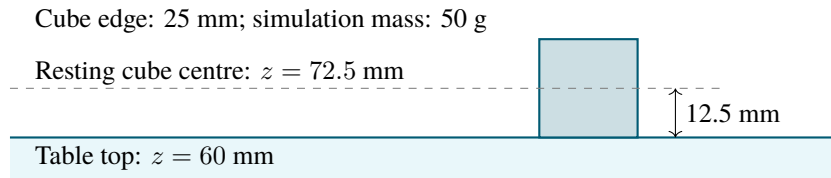


Figure 1. Height consistency in the world frame; schematic, not to scale.

The fixed object-centre targets are $A = (0.278, -0.001, 0.0725)$ m and $B = (0.237, 0.1453, 0.0725)$ m. The A/B markers are visual references, not collision platforms. Therefore, their appearance must not be interpreted as two elevated support pads. Contact with the actual table is checked when accepting a placement.

2.2 Mass and inertia

The simulation assigns mass $m = 0.05$ kg to the cube. For a uniform cube of edge $a = 0.025$ m, the principal moments are

$$I_{xx} = I_{yy} = I_{zz} = \frac{ma^2}{6} = 5.2083 \times 10^{-6} \text{ kg m}^2. \quad (1)$$

This is a model parameter, not a measured mass of the real object. The handout limits the real target to 100 g, but no measured physical mass is supplied. I would not claim that the 50 g simulation setting proves compliance of an unweighed real target.

2.3 Frames matter as much as dimensions

The kinematic model incorporates a 0.060 m base-height offset. Its home/hover/grasp tip coordinates are consequently not raw clearances above the tabletop. Mixing these coordinates with world object-centre coordinates, or with SDK millimetres, would create a systematic height error even if every numerical value were copied correctly.

3 Model resources and integration

3.1 Visual meshes, collision meshes and link frames

The robot model uses the community `robomaster_description` resources bundled with the project [4]. Visual geometry determines appearance; collision geometry determines physical interaction. The controller also relies on the model’s link/joint transforms to locate the fingertips. These three uses of the model must agree on frame origins and scale.

The supplied kinematics explicitly identifies the forward end of gripper link 4 as the grasping region; link 7 is a rear linkage rather than a fingertip. This distinction explains why choosing a link by its number or visual prominence is unsafe. An apparently reasonable end-effector reference can still be located at the wrong part of the mechanism.

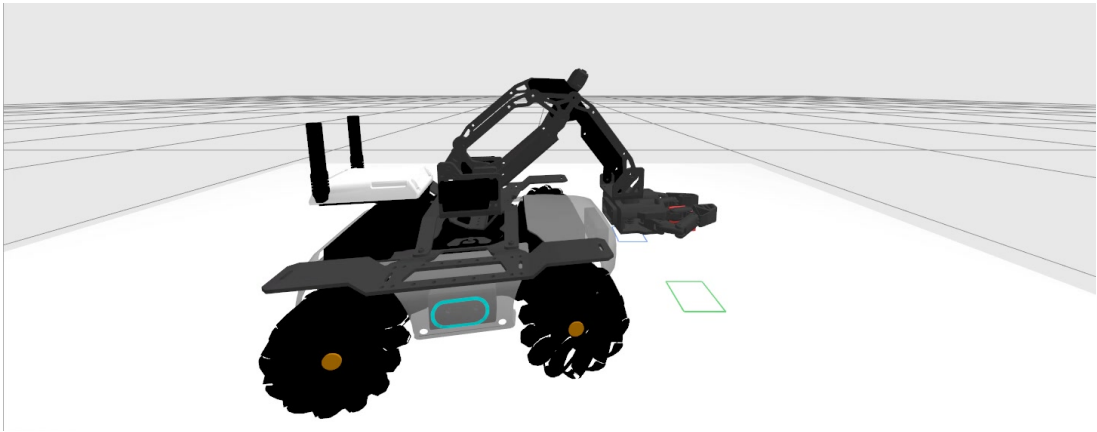


Figure 2. Scene frame from the team’s simulation recording at 34 s. The image documents model context, not real-hardware accuracy.

3.2 Making the archived model portable

Earlier development notes describe missing mesh resources and fragmented model appearance. The final archive includes the complete dependency directory and `prepare_paths.py`, which creates a runtime copy with resource paths adjusted to the bundled model. This is more reproducible than assuming another computer has the author’s original absolute Linux paths.

Component	Function and dependency
<code>ep_feedback.world.sdf</code>	Table, target cube, markers and world systems.
<code>ep_feedback.sdf</code>	Robot links/joints, collisions and control plugins.
<code>robomaster_description</code>	Mesh resources required by the archived model.
<code>ep_kinematics.py</code>	Frame reconstruction and fingertip-centre calculation.
<code>ep_transfer.launch.py</code>	Starts simulator, bridge, spawn and task nodes.

3.3 Starting the complete simulation

The startup wrapper sources ROS 2 Humble and the workspace environment. One launch file starts Gazebo, the bridge and refill node, schedules robot creation after 5 s and starts the task controller after 8 s. When preparing a run, the important checks are that the robot and cube have loaded and that joint/pose feedback is available. The fixed startup delays are not themselves proof of readiness; slow model loading can still prevent a task from starting.

4 Physical validity and observed results

4.1 Contact grasping versus scene manipulation

The final transfer controller uses gripper/object contact rather than continuously attaching or teleporting the cube along a scripted trajectory. Its bilateral-contact test identifies fresh collisions from both gripper sides. The cube must subsequently rise above the configured height condition before transfer continues.

There is nevertheless a legitimate object-position reset in the system: the separate refill node moves the cube back to A after home verification, between cycles 1–4. It is enabled only in REFILL_WAIT. This operation represents experiment replenishment, not the physical grasping mechanism. Keeping that distinction explicit avoids claiming that every object movement in the world is unmodified physics.

Cycle	Error (mm)	Heading (°)	Time (s)	Placed	Home
1	2.274	0.445	67.01	Yes	Yes
2	5.604	0.329	59.72	Yes	Yes
3	4.192	0.404	60.92	Yes	Yes
4	4.577	0.459	61.09	Yes	Yes
5	5.144	0.405	61.29	Yes	Yes

Source: Recorded simulation run 20260913_153855_682865/results.json; error is horizontal distance from configured B, heading is absolute home-heading error.

The recorded run completed five placements and five returns home. The maximum horizontal placement error was 5.604 mm; the configured acceptance limit is 23 mm, together with an 8 mm height window and table contact. The settled centre heights near 72.5 mm are consistent with the table-plus-half-cube calculation, providing a useful cross-check of the scene geometry.

These results support a functioning contact-based demonstration under the selected model and parameters. They do not establish mesh-level collision accuracy, realistic friction over all surfaces, or agreement with measured hardware dynamics. A successful placement is a system-level outcome, not an independent validation of every model parameter.

4.2 Physical validation: team-observed outcome

The team confirmed that all five hardware transfers succeeded with no observed abnormalities. Cycles 1, 3 and 5 moved A→B; cycles 2 and 4 moved B→A. Each lift was followed by a fresh human y confirmation before rotation; final heading restoration and homing followed cycle 5. This is an observed 5/5 result, not an instrumented precision or timing measurement. No real-hardware position-error, duration or force data are available, and five successes do not establish long-run reliability.

5 Problems, solutions and individual reflection

5.1 A complete-looking model can still be wrong

Missing mesh paths explain invisible parts, while incorrect frame or inertia references can produce a model that renders but behaves incorrectly. The archived solution is a complete resource set and explicit frame reconstruction. My preferred diagnostic order is to check resource resolution, dimensions, frame relationships and physical contact separately. Changing controller gains before checking those foundations risks compensating for a geometry error. The development notes also record pasted shell formatting and Python-environment problems. Keeping commands in plain scripts, sourcing the intended ROS environment and using the system interpreter helps separate an environment failure from a motion-program failure. This is the handoff point between my setup work and Huang Junbiao's program debugging.

5.2 Placement regions must match reachable geometry

Development notes identify unsuitable early placement locations, including a destination conflicting with the vehicle region. The final A/B arrangement uses chassis turning to bring the gripper toward B. A marker's presence is not enough: the reachable fingertip set, chassis footprint and descent corridor must all be compatible. A top-view geometric check should precede a full pick-and-place trial.

5.3 Clearance checking has a limited scope

The retrospective materials describe a nine-point geometric audit from $(x, z) = (0.235, 0.220)$ m to $(0.270, 0.160)$ m at closure 0.6, with no reported intersections at those samples. This is useful local evidence. However, the actual controller interpolates joint angles, so a straight Cartesian sample set is not necessarily the executed path. It also does not examine every link configuration during closing, turning or lowering.

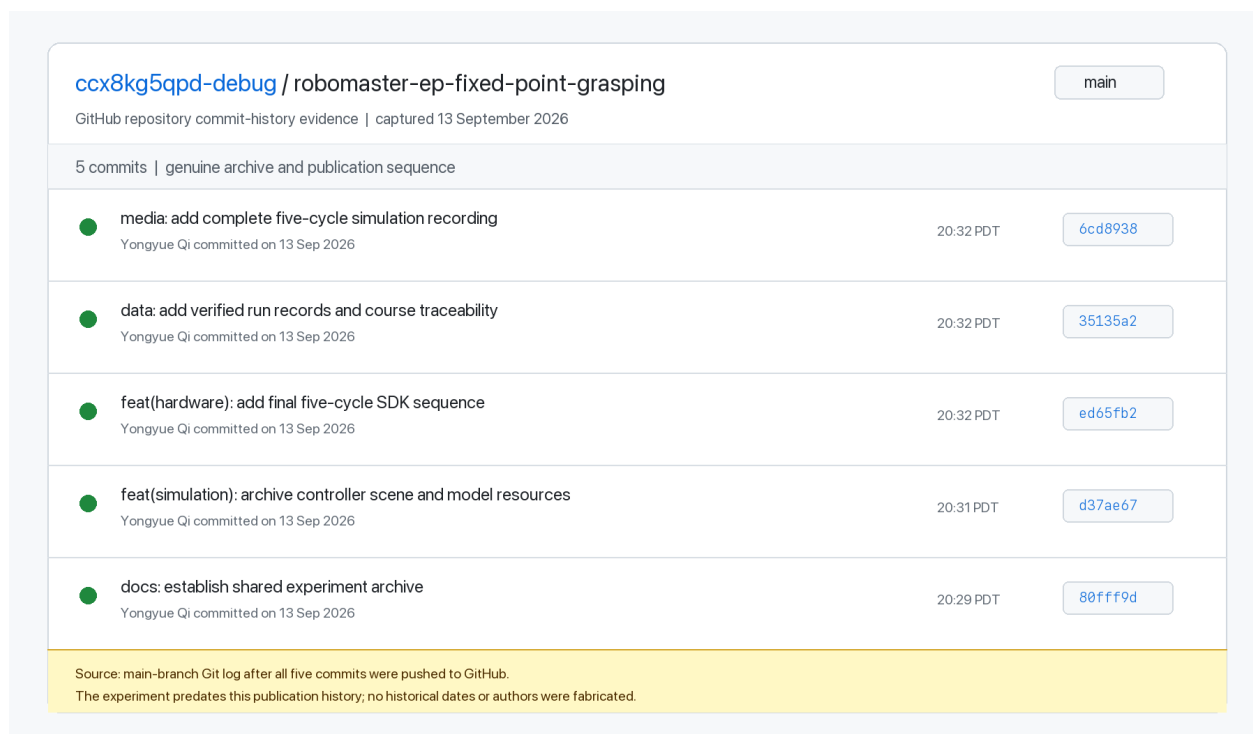
My proposed improvement is to evaluate the actual interpolated joint sequence against the collision model, then repeat with small target and object-size perturbations. Those checks should be reported as additional tests when performed, not retroactively asserted as completed. This would connect static scene validation to the moving mechanism more directly.

5.4 Main lesson

Scene building is an engineering task, not just visualization. Consistent support height, fingertip reference, mass/inertia and resource packaging make the downstream controller interpretable. The final five-cycle results are encouraging, but real-world transfer requires measured object properties and hardware calibration. For this report, I keep the 50 g mass, numerical contact behavior and millimetre errors explicitly within the simulation evidence.

6 Shared GitHub repository and commit evidence

The complete group archive is stored in the [shared private GitHub repository](#). The repository address is `github.com/ccx8kg5qpd-debug/robomaster-ep-fixed-point-grasping`. It contains the simulation and hardware programs, model resources, saved results, the complete simulation recording and all five individual report sources. The real-hardware video remains explicitly marked as pending and will be added after recording.



ccx8kg5qpd-debug / robomaster-ep-fixed-point-grasping			
GitHub repository commit-history evidence captured 13 September 2026			
5 commits genuine archive and publication sequence			
media: add complete five-cycle simulation recording Yongyue Qi committed on 13 Sep 2026	20:32 PDT	6cd8938	
data: add verified run records and course traceability Yongyue Qi committed on 13 Sep 2026	20:32 PDT	35135a2	
feat(hardware): add final five-cycle SDK sequence Yongyue Qi committed on 13 Sep 2026	20:32 PDT	ed65fb2	
feat(simulation): archive controller scene and model resources Yongyue Qi committed on 13 Sep 2026	20:31 PDT	d37ae67	
docs: establish shared experiment archive Yongyue Qi committed on 13 Sep 2026	20:29 PDT	80fff9d	

Source: main-branch Git log after all five commits were pushed to GitHub.
The experiment predates this publication history; no historical dates or authors were fabricated.

Figure 3. Commit-history evidence captured after the first five project materials commits had been pushed to the shared GitHub repository.

Blueprint note. Traceability note. The experiment was completed before the GitHub repository was assembled. These genuine commits document the later review, organization and publication sequence; they are not presented as a fabricated contemporaneous development history. The screenshot records the five commits visible at capture time, before this revised report was added.

Sources and traceability

- [1] Course handout, *Fixed-Point Robotic-Arm Grasping: Simulation Development and Hardware Validation*, S16–S20 / E5; individual-report assignment requirements.
- [2] Shared team repository, [robomaster-ep-fixed-point-grasping](#), assembled 13 September 2026: simulation source/configuration, `real_v2_five_cycles_manual.py`, saved logs and reports. Primary numerical record: 20260913_153855_682865.
- [3] Team conversation fact-check materials, 13 September 2026, for development history; subsequent team confirmation for the completed five-cycle hardware run. Historical failures are not counted as final-run failures.
- [4] Community model resources: [jeguZZi/robomaster_ros](#), archived commit c05a39d7f0fa. Report layout uses [ElegantPaper](#), distributed under LPPL 1.3c.